Chapter 4. Dynamic objects

This chapter covers

- Objects as data
- Object comprehensions
- An introduction to prototypes
- An introduction to classes

JavaScript objects are simple and powerful, but because they don't work quite like objects in other popular programming languages, they're often wrongly perceived as confusing and underpowered. With simplified and familiar syntax, CoffeeScript both eliminates the confusion and better exposes the inherent power of objects. Before diving in, though, what exactly is an object?

Objects are collections of named properties where each name maps to a value. The value of a property can be any valid CoffeeScript value. You write an empty object with a set of curly braces:

```
{}
```

That is an object, in *literal notation*. The literal notation for objects is small and convenient. It's used in the source code of programs to represent data and as a format for transferring data on the web.

Beyond being simple property containers, objects are also used for managing state and structuring programs. Objects in CoffeeScript are based on a concept called *prototypes*, which have the advantages of being dynamic and flexible but the disadvantages of being uncommon in other programming languages and unfamiliar to most programmers. It's classes, and not prototypes, that are a familiar concept in other programming languages, so CoffeeScript provides class syntax that allows programs to be structured using more familiar class-based techniques.